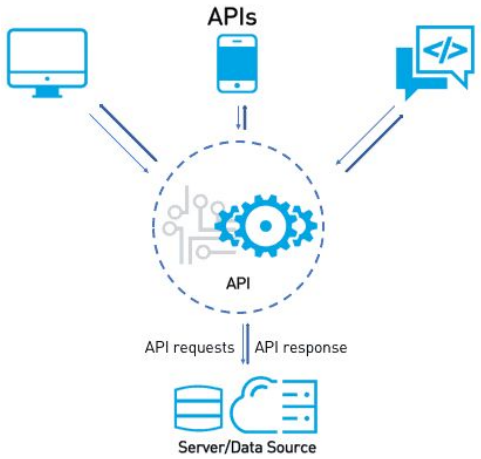
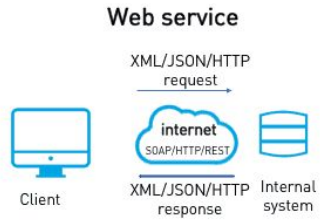
Web Services Development

Resource folder: shorturl.at/tAEMN





Rwanda Coding Academy

by damascene10

Table of Content

Introduction to web services

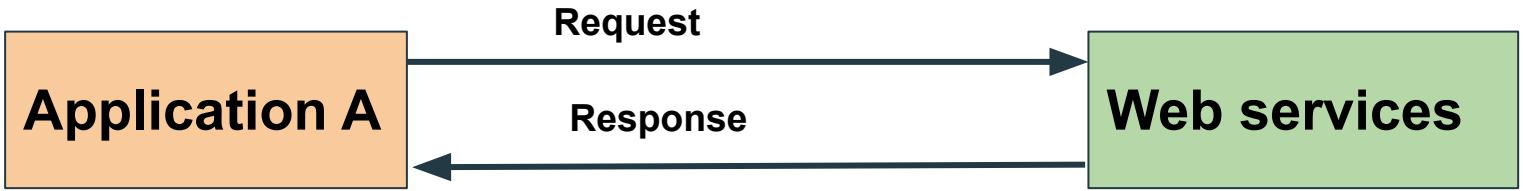
1.0 What is web service

1. Web services are the types of internet software that uses standardized messaging protocol over the distributed environment.
2. A Web service can also be defined as a self-contained, self-describing, modular application, based on XML/JSON that can be published to and invoked from the Web.
3. A service offered by an electronic device to another electronic device, communicating with each other via the World Wide Web.
4. A server running on a computer device, listening for requests at a particular port over a network, serving web documents (HTML. XML,JSON).

It integrates the web-based application using the **REST(**Representational state transfer), **SOAP(Simple Object Access Protocol), WSDL(Web Services Description Language)**, and **UDDI**(Universal Description, Discovery, and Integration)over the network. For example, Java web service can communicate with .Net application.

Suppose, we have an **Application A** which create a request to access the web services. The web services offer a list of services. The web service process the request and sends the response to the **Application A**.

The input to a web service is called a request, and the output from a web service is called response. The web services can be called from different platforms.



There are two popular formats for request and response **XML** and **JSON.**

Web Services and APIs

While APIs and web services can both facilitate data transfers between applications over the internet, they are not the same, and the terms should not be used interchangeably in every case. The key distinction is that web services are a type of API: All web services are APIs, but not all APIs are web services.

API is the acronym for Application Programming Interface. It is a software interface that allows two applications to interact with each other without any user intervention.

Since web services are designed to share data with other disconnected applications, this qualifies them as APIs. However, a web service is just one way to implement an API.

XML

XML(eXtensible Markup Language) Format: XML is the popular form as request and response

in web services. Consider the following XML code:

```
<getDetail>
    <id>DataStructureCourse</id>
</getDetail>
```

The code shows that user has requested to access the DataStrutureCourse.

JSON

The other data exchange format is JSON. JSON is supported by wide variety of platform.

JSON(JavaScript Object Notation) Format: JSON is a readable format for structuring data. It is used for transiting data between server and web application

```
[
"employee":
{
"id": 00987
"name": "Jack",
"salary": 20000,
}
```

To make a web service platform independent, we make the request and response platform-independent.

Now a question arises, how does the Application A know the format of Request and Response?

The answer to this question is "Service Definition." Every web service offers a service definition.

Service definition specifies the following:

o**Request/ Response format:** Defines the request format made by consumer and response format made by web service.

o Request Structure: Defines the structure of the request made by the application.

o Response Structure: Defines the structure of response returned by the web service.

o Endpoint: Defines where the services are available.

1.1 Key terminologies

Request and Response: Request is the input to a web service, and the response is the output from a web service.

Message Exchange Format: It is the format of the request and response. There are two popular message exchange formats: XML and JSON.

Service Provider or Server: Service provider is one which hosts the web service.

Service Consumer or Client: Service consumer is one who is using the web service.

Service Definition: Service definition is the contract between the service provider and service consumer. Service definition defines the format of request and response, request structure, response structure, and endpoint.

Transport: Transport defines how a service is called. There is two popular way of calling a service: **HTTP** (Hypertext Transfer Protocol) and Message Queue (**MQ**). By tying the URL of service, we can call the service over the internet. MQ communicates over the queue. The service requester puts the request in the queue. As soon as the service provider listens to the request. It takes the request, process the request, and create a response, and put the response back into MQ.

The service requester gets the response from the queue. The communication happens over the

Queue.

Service Reuse

Takes code reuse a step further. A specific function within the domain is only ever coded once and used repeatedly by consuming applications.

1.2 Features of web Services

- Web services are designed for application to application interaction.
- It should be interoperable. Not platform dependent
- It should allow communication over the network.

1.3 Uses of Web Services

- **Loosely Coupled**

Each service exists independently of the other services that make up the application. Individual pieces of the application can be modified without impacting unrelated areas.

- **Ease of Integration**

Data is isolated between applications creating the ecosystem. Web Services act as glue between these and enable easier communications within and across organizations.

1.4 Types of Web Services

There are two types of web services:

- **RESTful Web Services**
- **SOAP Web Services**

Microsoft developed SOAP as a web communication protocol. One of its most important features is that it is platform-independent. It can also operate over various protocols such as HTTP (Hypertext Transfer Protocol), SMTP (Simple Mail Transfer Protocol), TCP (Transmission Control Protocol) or UDP (User Datagram Protocol).

Historically, SOAP was the most commonly used option. SOAP exclusively uses **XML**

(Extensible Markup Language). It is standardized and follows strict rules, making it a rather inflexible but highly structured option. As a result, it is very well-documented.

1.4.1 RESTful Web Services

REST stands for **RE**presentational **S**tate **T**ransfer. It is developed by **Roy Thomas Fielding** who developed **HTTP** as well. The main goal of RESTful web services is to make web services more effective. RESTful web services try to define services using the different concepts that are already present in HTTP. REST is an **architectural approach**, not a protocol.

It does not define the standard message exchange format. We can build REST services with both XML and JSON. JSON is more popular format with REST. **The key abstraction** is a resource in REST. A resource can be anything. It can be accessed through a **Uniform Resource Identifier (URI).**

The resource has representations like XML, HTML, and JSON. The current state is captured by

representational resource. When we request a resource, we provide the representation of the

resource. The important methods of HTTP are:

- **GET:** It reads a resource.
- **PUT**: It updates an existing resource.
- **POST**: It creates a new resource.
- **DELETE**: It deletes the resource.

For example, if we want to perform the following actions in the social media application, we get the corresponding results.

POST /users: It creates a user.

GET /users/{id}: It retrieve the detail of one user.

GET /users: It retrieve the detail of all users.

DELETE /users: It delete all users.

DELETE /users/{id}: It delete a user.

GET /users/{id}/posts/post_id: It retrieve the detail of a specific post.

POST / users/{id}/ posts: It creates a post for a user.

GET /users/{id}/post: Retrieve all posts for a user

1.4.1.1 RESTful status Code

HTTP also defines the following standard status code:

- **404**: RESOURCE NOT FOUND
- **200**: SUCCESS
- **201**: CREATED
- **401**: UNAUTHORIZED
- **500**: SERVER ERROR

RESTful Service Constraints

- There must be a service **producer** and service **consumer**.
- The service is stateless.
- The service result must be cacheable.
- The interface is uniform and exposing resources.
- The service should assume a layered architecture.

1.4.1.2 Advantages of RESTful web services

- RESTful web services are platform-independent.
- It can be written in any programming language and can be executed on any platform.
- It provides different data format like JSON, text, HTML, and XML.
- It is fast in comparison to SOAP because there is no strict specification like SOAP.
- These are reusable.
- These are **language neutral**.
- Requires less bandwidth than SOAP
- All of these advantages have meant that Google and many other such well-known giants use

REST, giving it the good name and popularity that it has today.

1.4.1.3 REST Cons: REST vs SOAP

- Greater margin for error in greater flexibility
- No built-in stateful capabilities
- You may require additional employee hours of development time in order to provide benefits that SOAP has built-in with the WS standards

1.4.1.4 RestFul services use

While you can use REST to produce many if not all of the same results as SOAP with enough work from a professional developer, there are a few situations that REST is specially geared towards.

Limited bandwidth, low-cost or simple operations. Why take the time to write complex XML when you can work from a simple URL to perform a basic, non-repetitive task?

Additionally, if you have a lower income business at the moment or limited bandwidth, REST will perform much faster and give better results in these situations than SOAP.

Developing public APIs. REST is a useful framework for developing public APIs because it is so common, and therefore it is easy to find other developers who understand

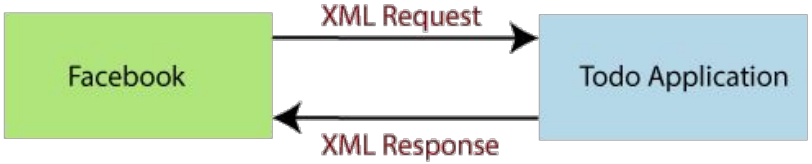
it. Additionally, it is highly flexible, bringing lots of options to the marketplace. One example is Gmail, which is a RESTful API. Github features public APIs from various developers for various purposes, including health, jobs, cryptocurrency, machine learning, anti-malware and even the weather, just to name a few. You can also use REST APIs to import or export content, create and manage credentials and much more.

1.4.2. SOAP Web services

REST defines an architectural approach whereas SOAP poses **a restriction on the format of the XML.** XML transfer data between the service provider and service consumer. Remember that SOAP and REST are not comparable.

SOAP: SOAP acronym for **Simple Object Access Protocol**. It defines the standard XML format. It also defines the way of building web services. We use Web Service Definition Language (**WSDL**) to define the format of **request XML** and the **response XML.**

For example, we have requested to access the Todo application from the Facebook application. The Facebook application sends an XML request to the Todo application. Todo application processes the request and generates the XML response and sends back to the Facebook application.



XML Resources

- https://www.tutorialspoint.com/xml/index.htm
- https://www.javatpoint.com/xml-tutorial
- https://www.w3schools.com/xml/

1.4.2.1.Key Points to remember

- SOAP defines the format of request and response.
- SOAP does not pose any restriction on transport. We can either use HTTP or MQ for communication.
- In SOAP, service definition typically done using Web Service Definition Language

  (WSDL). **WSDL** defines **Endpoint, All Operations, Request Structure, and Response Structure.**

  The **Endpoint** is the connection point where HTML or ASP pages are exposed. It provides the information needed to address the Web Service endpoint. The operations are the services that are allowed to access. Request structure defines the structure of the request, and the response structure defines the structure of the response.

1.4.2.2 Web Service Description Language (WSDL)

WSDL acronym for Web Service Description Language. **WSDL** is an XML based interface description language. It is used for describing the functionality offered by a web service.

Sometimes it is also known as the **WSDL file**. The extension of the WSDL file is **.wsdl**. It provides the machine-readable description of how the service can be called, what parameter it expects, and what data structure it returns.

It describes service as a collection of network endpoint, or ports. It is often used in combination with SOAP and an XML schema to provide XML service over the distributed environment. In short, the purpose of **WSDL** is similar to type-signature in a programming language.

1.4.2.3 SOAP Pros: REST vs. SOAP

- Strong documentation
- Strict regulations, allowing for increased accuracy and easy collaboration
- Includes built-in WS (Web Services) Security
- Includes built-in extensibility through various other WS standards
- Is ideal for formal contracts due to its in-depth regulations
- Made to support stateful operations
- Various transport options

1.4.2.4 SOAP Cons: REST vs SOAP

- SOAP does have some very specific drawbacks that have encouraged companies to seek a different solution.
- Complexity
- Tends to be slow
- Only uses XML

These things, and the fact that many big names use **REST**, have caused SOAP to fall into the background somewhat. Nonetheless, there are still instances where SOAP can be very useful, as you will see later in the examples.

1.4.2.5 SOAP USAGE Examples

When should you use SOAP? SOAP is best for anything that requires formal contracts. To be even more specific, here are two common use cases for SOAP.

1. Asynchronous operations. An asynchronous operation is very time-specific. It is when various signals or preceding events trigger new events, rather than an external timer. Asynchronous means they are totally independent and neither one must consider the other in any way, either in the initiation or in execution.

REST limits itself to HTTP and HTTPS, neither of which are the ideal communication protocols for this purpose as they may delay such an operation. SOAP supports additional communication protocols. Asynchronous operations are not suited either for very simple tasks or exceptionally complex ones, since it can be difficult to maintain correct timing.

SYNCHRONOUS VS ASYNCHRONOUS EXAMPLE

Suppose you are in a queue to buy an entrance ticket. You cannot get one until everybody in front of you gets one, and the same applies to the people queued behind you. That is Synchronous. Now consider several gates for selling tickets. Therefore, each line can work asynchronously from the other line but synchronously within itself!

2. Stateful operations. Performing repetitive, chained tasks such as the financial industry requires means that you may need to retain certain client data within the server for future use. *In real life think of stateful transactions as an ongoing periodic conversation with the same person.*

By default**, REST is stateless.** In **stateless** operation,there is no stored knowledge of or reference to past transactions. Each transaction is made as if from scratch for the first time. RESTful apps will not save any previous transactions, but SOAP supports stateful operations.

There are many other such use cases for SOAP as well, but most have to do with some variation of these two operations.

1.4.3. Summarised Difference: REST VS SOAP

SOAP	**REST**
SOAP is a protocol.	REST is an architectural approach.
SOAP is an acronym for **S**imple **O**bject **A**ccess **P**rotocol	**REST** acronym for **RE**presentational **S**tate **T**ransfer.
In SOAP, the data exchange format is always **XML**.	There is no strict data exchange format. We can use **JSON**, **XML**, etc.
XML is the most popular data exchange format in SOAP web web services.	JSON is the most popular data exchange format in RESTful services
SOAP does not pose any restrictions on transport. We can use either HTTP or MQ	RESTful services use the most popular HTTP protocol.

1.4.3. Summarised Difference: REST VS SOAP

SOAP	REST
SOAP web services are typical to implement	RESTful services are easier to implement than SOAP.
SOAP web services use the JAX-WS API	RESTful web services use the JAX-RS API.
SOAP protocol defines too many standards	RESTful services do not emphasis on too many standards.
SOAP cannot use RESTful services because it is a protocol	RESTful service can use SOAP web services because it is an architectural approach that can use any protocol like HTTP and SOAP.
SOAP reads cannot be cached.	REST reads can be cached.